

CalcScript

Analizador Léxico y Sintáctico

Proyecto de Programación de Sistemas Base 1

Campo	Valor
Materia	Programación de Sistemas Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor	Muñoz Quintero Dante Adolfo
Integrantes	Romero Guzmán Martín Gastón Mar Saucedo Duilio Alessandro
Repositorio	https://github.com/martinroguz-netizen/calcscrip-lexer
Fecha	Mayo 2026

El presente documento constituye la Demostración Funcional del proyecto de semestre. Incluye la especificación del lenguaje, el código fuente del analizador léxico y sintáctico (que genera AST como representación intermedia), el diagrama del autómata léxico, y capturas de pantalla reales de la ejecución.

Tabla de contenido

1. Introducción
2. Objetivos
3. Especificación del Lenguaje
 - 3.1 Nombre y propósito
 - 3.2 Ejemplos ilustrativos
 - 3.3 Catálogo de tokens
 - 3.4 Gramática sintáctica (EBNF)
4. Implementación
 - 4.1 tokens.py
 - 4.2 rules.py
 - 4.3 lexer.py
 - 4.4 ast_nodes.py
 - 4.5 parser.py
 - 4.6 main.py
5. Diagrama del Autómata Léxico
6. Demostración Funcional (capturas de pantalla)
 - 6.1 Programa válido: factorial.csc
 - 6.2 Errores léxicos: errores_simbolos.csc
 - 6.3 Errores sintácticos: errores_sintaxis.csc
 - 6.4 Errores mixtos: errores_mixtos.csc
7. Conclusiones
8. Referencias

1. Introducción

Un compilador procesa código fuente en varias fases. Las dos primeras son el **análisis léxico** (caracteres $\rightarrow$ tokens) y el **análisis sintáctico** (tokens $\rightarrow$ AST). Este proyecto implementa ambos componentes para **CalcScript**, un lenguaje educativo para cálculos aritméticos con variables, estructuras de control y funciones.

El lexer usa expresiones regulares (re) y el parser implementa **descenso recursivo** manual con recuperación de errores por sincronización. Todo en Python 3.12 sin dependencias externas.

2. Objetivos

Objetivo general

Diseñar **CalcScript** e implementar su analizador léxico y sintáctico, generando un AST como representación intermedia, con manejo de errores léxicos y sintácticos.

Objetivos específicos

1. Definir la especificación del lenguaje con ejemplos ilustrativos.
2. Establecer un catálogo completo de tokens.
3. Implementar el lexer con expresiones regulares ordenadas.
4. Formalizar la gramática sintáctica en EBNF con precedencia de operadores.
5. Implementar un parser por descenso recursivo que genere AST.
6. Implementar recuperación de errores léxicos y sintácticos con línea y columna.
7. Construir el diagrama del autómata finito determinista (DFA) del lexer.
8. Probar con programas válidos y con errores léxicos, sintácticos y mixtos.

3. Especificación del Lenguaje

3.1 Nombre y propósito

CalcScript es un lenguaje imperativo educativo para cálculos aritméticos, asignaciones, `if/else`, `while` y funciones. Extensión: `.csc`.

3.2 Ejemplos ilustrativos

Ejemplo 1 — Factorial:

```
func factorial(n) {
    result = 1;
    while n > 1 {
        result = result * n;
        n = n - 1;
    }
    return result;
}

x = factorial(5);
print(x);
```

Ejemplo 2 — Condicional:

```
a = 3.14;
b = 2.0;

if a > b {
    diff = a - b;
    print(diff);
} else {
    print(b);
}
```

Ejemplo 3 — Aritmética:

```
total = (10 + 5) * 2 / 3;
print(total);
```

3.3 Catálogo de tokens

Categoría	Ejemplos	Descripción
KEYWORD	if, else, while, func, return, print	Palabras reservadas
IDENTIFIER	x, total, my_var	Letra/_ + letras/dígitos/_
NUMBER_INT	0, 42, 1024	Dígitos sin punto
NUMBER_FLOAT	3.14, 0.5, 2.0	Dígitos.dígitos
OPERATOR	+, -, *, /, =, >, <, >=, <=, ==, !=	Aritméticos y relacionales
DELIMITER	;, () {}	Separadores
STRING	"hola"	Cadena entre comillas dobles

COMMENT	# comentario	Hasta fin de línea (descartado)
---------	--------------	---------------------------------

Errores léxicos: identificadores que comienzan con dígito y símbolos desconocidos. Cada error con línea, columna y descripción.

3.4 Gramática sintáctica (EBNF)

```

program      = { statement } ;
statement    = funcDecl | ifStmt | whileStmt | returnStmt
              | printStmt | block | assignment | exprStmt ;
funcDecl     = "func" IDENT "(" [paramList] ")" block ;
ifStmt       = "if" expression block [ "else" block ] ;
whileStmt    = "while" expression block ;
returnStmt   = "return" [ expression ] ";" ;
printStmt    = "print" "(" expression ")" ";" ;
assignment   = IDENT "=" expression ";" ;
block        = "{" { statement } "}" ;

expression   = equality ;
equality      = comparison { ("==" | "!=") comparison } ;
comparison   = term { (">" | "<" | ">=" | "<=") term } ;
term         = factor { ("+" | "-") factor } ;
factor       = unary { ("*" | "/") unary } ;
unary        = "-" unary | call ;
call         = primary [ "(" [argList] ")" ] ;
primary      = NUMBER_INT | NUMBER_FLOAT | STRING
              | IDENT | "(" expression ")" ;

```

4. Implementación

Código fuente organizado en seis módulos con responsabilidades separadas. Disponible completo en el repositorio GitHub.

4.1 tokens.py

Enumeración de tipos de token y palabras reservadas.

```
# tokens.py
# Define los tipos de token reconocidos por CalcScript y las palabras
# reservadas que no pueden usarse como identificadores.
#
# Usar una clase de constantes centralizadas (Enum) garantiza que el lexer
# y futuras fases del compilador compartan exactamente los mismos
# identificadores de tipo, evitando errores de comparación por strings.

from enum import Enum, auto

class TokenType(Enum):
    """Tipos de token reconocidos por el analizador léxico de CalcScript."""

    # Literales
    NUMBER_INT = auto()      # entero: 42, 0, 1024
    NUMBER_FLOAT = auto()    # flotante: 3.14, 0.5
    STRING = auto()          # cadena: "hola"
    IDENTIFIER = auto()      # nombre de variable o función

    # Palabras reservadas (se detectan reclasificando IDENTIFIER)
    KEYWORD = auto()

    # Operadores: + - * / = > < >= <= == !=
    OPERATOR = auto()

    # Delimitadores: ; , ( ) { }
    DELIMITER = auto()

    # Control interno
    EOF = auto()             # fin de archivo
    ERROR = auto()           # token inválido (se reporta y se descarta)

# Conjunto de palabras reservadas. Se usa para reclasificar
# un IDENTIFIER reconocido en un KEYWORD cuando coincide con esta tabla.
KEYWORDS: set[str] = {
    "if", "else", "while", "func", "return", "print",
}
```

4.2 rules.py

Expresiones regulares ordenadas del lexer.

```
# ■■ rules.py ■■■
# Pares (TokenType, regex) evaluados en orden por el lexer principal.
#
# IMPORTANTE: el orden de las reglas es crítico. Las reglas más específicas
# o más largas deben evaluarse ANTES que las generales:
#   - NUMBER_FLOAT antes que NUMBER_INT (para que 3.14 no se tokenice 3,.,14)
#   - Operadores de 2 caracteres antes que los de 1 (>= antes que >)
#
# Las reglas cuyo TokenType es None se reconocen y descartan
# (espacios en blanco y comentarios).

import re
from tokens import TokenType

# Cada entrada: (tipo, patrón compilado)
RULES: list[tuple[TokenType | None, re.Pattern]] = [

    # ■■ Espacios en blanco y comentarios → se descartan (None = ignorar) ■■
    (None, re.compile(r'[ \t\r]+')),          # espacios y tabuladores
    (None, re.compile(r'#[^\n]*')),          # comentario hasta fin de línea

    # ■■ Literales numéricos ■■
    # FLOAT obligatoriamente con dígitos a ambos lados del punto
    (TokenType.NUMBER_FLOAT, re.compile(r'\d+\.\d+')),
    (TokenType.NUMBER_INT, re.compile(r'\d+')),

    # ■■ Cadenas de texto ■■
    # Cadena entre comillas dobles; admite cualquier carácter excepto " y \n
    (TokenType.STRING, re.compile(r'"[^\n]*"')),

    # ■■ Identificadores ■■
    # Letra o '_' seguido de letras, dígitos o '_'.
    # El lexer luego reclasifica a KEYWORD si el lexema coincide con la tabla.
    (TokenType.IDENTIFIER, re.compile(r'[A-Za-z_][A-Za-z_0-9]*')),

    # ■■ Operadores de dos caracteres (antes que los de uno) ■■
    (TokenType.OPERATOR, re.compile(r'==|!=|>|=|<=|<|>')),

    # ■■ Operadores de un solo carácter ■■
    (TokenType.OPERATOR, re.compile(r'[+\-*/=<>]')),

    # ■■ Delimitadores ■■
    (TokenType.DELIMITER, re.compile(r'[;,(){}]*')),
```


Bucle principal del analizador léxico.

```
# lexer.py
# Clase principal del analizador léxico de CalcScript.
# Recibe el texto fuente y produce una lista de Token y una lista de errores.
#
# Estrategia de escaneo:
# 1. Mantiene un cursor `pos` sobre el texto fuente y un par (line, column).
# 2. En cada iteración intenta hacer coincidir las reglas de rules.py en
#    orden, desde la posición actual.
# 3. Si la regla devuelve un tipo None (espacio o comentario), se descarta.
# 4. Si reconoce IDENTIFIER, verifica si el lexema es palabra reservada
#    (KEYWORDS) y reclasifica el TokenType a KEYWORD.
# 5. Si reconoce NUMBER_INT, verifica si el carácter siguiente es letra
#    o '_': eso indica un identificador inválido que comienza con dígito
#    (caso 123abc). El número se emite y se reporta el error.
# 6. Si ninguna regla coincide, emite un LexError con la posición exacta
#    y avanza UN carácter, para evitar bucles infinitos y poder seguir
#    analizando el resto del archivo.

from dataclasses import dataclass

from tokens import TokenType, KEYWORDS
from rules import RULES

@dataclass
class Token:
    """Unidad léxica reconocida."""
    index: int          # número secuencial (1-based)
    type: TokenType
    value: str
    line: int
    column: int

    def __str__(self) -> str:
        return f"{self.index}: ({self.type.name}, \"{self.value}\")"

@dataclass
class LexError:
    """Error léxico con ubicación exacta."""
    line: int
    column: int
    message: str

    def __str__(self) -> str:
        return f"Line {self.line}, Column {self.column}: {self.message}"

class Lexer:
    """Analizador léxico de CalcScript."""

    def __init__(self, source: str) -> None:
        self.source: str = source
        self.pos: int = 0          # índice absoluto sobre self.source
        self.line: int = 1         # línea actual (1-based)
        self.column: int = 1       # columna actual (1-based)
        self.tokens: list[Token] = []
        self.errors: list[LexError] = []
```

[illegible]

```

        return False

def _emit(self, tok_type: TokenType, lexeme: str,
          line: int, column: int) -> None:
    """Agrega un token a la lista, asignándole índice secuencial."""
    self._token_index += 1
    self.tokens.append(Token(
        index=self._token_index,
        type=tok_type,
        value=lexeme,
        line=line,
        column=column,
    ))

def _report_unknown_symbol(self, ch: str) -> None:
    """Registra un error por símbolo no reconocido."""
    self.errors.append(LexError(
        self.line, self.column,
        f"Unknown symbol '{ch}' detected",
    ))

def _advance(self, n: int) -> None:
    """Avanza n posiciones sin cruzar saltos de línea.

    El bucle principal ya maneja los '\n' antes de invocar reglas,
    por lo que aquí basta con incrementar columna y posición.
    """
    self.pos += n
    self.column += n

```

Nodos del AST como dataclasses + pretty-printer.

@dataclass


```

class Identifier:
    name: str


# Aliases de tipo para legibilidad
Stmt = (
    FuncDecl | IfStmt | WhileStmt | ReturnStmt | PrintStmt
    | Assignment | ExprStmt | Block
)
Expr = (
    BinaryOp | UnaryOp | Call | IntLiteral
    | FloatLiteral | StringLiteral | Identifier
)


# ████████████████████████████████████████ Pretty printer ████████████████████████████████████████
def ast_to_string(node, indent: int = 0) -> str:
    """Convierte un AST a string indentado, legible para humanos.

    Cada nivel de profundidad añade 2 espacios. Los nodos hoja
    (literales, identificadores) se imprimen en una sola línea.
    """
    pad = "  " * indent
    cls = type(node).__name__

    # ■■ Nodos hoja ■■
    if isinstance(node, IntLiteral):
        return f"{pad}IntLiteral({node.value})"
    if isinstance(node, FloatLiteral):
        return f"{pad}FloatLiteral({node.value})"
    if isinstance(node, StringLiteral):
        return f"{pad}StringLiteral({node.value!r})"
    if isinstance(node, Identifier):
        return f"{pad}Identifier({node.name!r})"

    # ■■ Nodos compuestos ■■
    if isinstance(node, Program):
        lines = [f"{pad}Program"]
        for s in node.statements:
            lines.append(ast_to_string(s, indent + 1))
        return "\n".join(lines)

    if isinstance(node, Block):
        lines = [f"{pad}Block"]
        for s in node.statements:
            lines.append(ast_to_string(s, indent + 1))
        return "\n".join(lines)

    if isinstance(node, FuncDecl):
        params = ", ".join(node.params)
        lines = [f"{pad}FuncDecl(name={node.name!r}, params=[{params}])"]
        lines.append(ast_to_string(node.body, indent + 1))
        return "\n".join(lines)

    if isinstance(node, IfStmt):
        lines = [f"{pad}IfStmt"]
        lines.append(f"{pad} condition:")
        lines.append(ast_to_string(node.condition, indent + 2))
        lines.append(f"{pad} then:")
        lines.append(ast_to_string(node.then_block, indent + 2))
        if node.else_block is not None:
            lines.append(f"{pad} else:")

```

```

        lines.append(ast_to_string(node.else_block, indent + 2))
    return "\n".join(lines)

if isinstance(node, WhileStmt):
    lines = [f"{pad}WhileStmt"]
    lines.append(f"{pad}  condition:")
    lines.append(ast_to_string(node.condition, indent + 2))
    lines.append(f"{pad}  body:")
    lines.append(ast_to_string(node.body, indent + 2))
    return "\n".join(lines)

if isinstance(node, ReturnStmt):
    if node.value is None:
        return f"{pad}ReturnStmt(no value)"
    lines = [f"{pad}ReturnStmt"]
    lines.append(ast_to_string(node.value, indent + 1))
    return "\n".join(lines)

if isinstance(node, PrintStmt):
    lines = [f"{pad}PrintStmt"]
    lines.append(ast_to_string(node.value, indent + 1))
    return "\n".join(lines)

if isinstance(node, Assignment):
    lines = [f"{pad}Assignment(name={node.name!r})"]
    lines.append(ast_to_string(node.value, indent + 1))
    return "\n".join(lines)

if isinstance(node, ExprStmt):
    lines = [f"{pad}ExprStmt"]
    lines.append(ast_to_string(node.expr, indent + 1))
    return "\n".join(lines)

if isinstance(node, BinaryOp):
    lines = [f"{pad}BinaryOp(op={node.op!r})"]
    lines.append(ast_to_string(node.left, indent + 1))
    lines.append(ast_to_string(node.right, indent + 1))
    return "\n".join(lines)

if isinstance(node, UnaryOp):
    lines = [f"{pad}UnaryOp(op={node.op!r})"]
    lines.append(ast_to_string(node.operand, indent + 1))
    return "\n".join(lines)

if isinstance(node, Call):
    lines = [f"{pad}Call(callee={node.callee!r})"]
    for arg in node.args:
        lines.append(ast_to_string(arg, indent + 1))
    return "\n".join(lines)

return f"{pad}{cls}{?}"

```

Parser recursivo descendente con recuperación de errores.

```
# parser.py
# Analizador sintáctico de CalcScript.
#
# Implementa un parser por descenso recursivo: una función Python por cada
# no-terminal de la gramática. Recibe la lista de tokens producida por el
# lexer y devuelve un Program (raíz del AST) más una lista de errores
# sintácticos.
#
# Gramática (EBNF):
#
# program      := statement*
# statement    := funcDecl | ifStmt | whileStmt | returnStmt | printStmt
#               | block | assignment | exprStmt
# funcDecl     := "func" IDENT "(" paramList? ")" block
# paramList    := IDENT ("," IDENT)*
# ifStmt       := "if" expression block ("else" block)?
# whileStmt    := "while" expression block
# returnStmt   := "return" expression? ";"
# printStmt    := "print" "(" expression ")" ";"
# assignment   := IDENT "=" expression ";"
# exprStmt     := expression ";"
# block        := "{" statement* "}"
#
# expression   := equality
# equality      := comparison (("==" | "!=") comparison)*
# comparison   := term ((">" | "<" | ">=" | "<=") term)*
# term         := factor (("+ | "-) factor)*
# factor       := unary (("*" | "/" ) unary)*
# unary        := "-" unary | call
# call         := primary "(" argList? ")"?
# argList      := expression ("," expression)*
# primary      := NUMBER_INT | NUMBER_FLOAT | STRING | IDENT
#               | "(" expression ")"
#
# Precedencia (de menor a mayor): == != < > <= >= < + - < * / < unario < ( )
#
# Recuperación de errores: cuando una regla falla, se reporta el error y se
# avanza hasta el siguiente ";" o palabra clave de inicio de sentencia, para
# poder seguir parseando y reportar más errores.

from dataclasses import dataclass

from lexer import Token
from tokens import TokenType
from ast_nodes import (
    Program, Block, FuncDecl, IfStmt, WhileStmt, ReturnStmt, PrintStmt,
    Assignment, ExprStmt,
    BinaryOp, UnaryOp, Call, IntLiteral, FloatLiteral, StringLiteral,
    Identifier,
)

# Excepción interna
class _ParseError(Exception):
    """Excepción usada solo internamente para hacer recuperación.

    No se propaga al usuario: el parser la captura y sincroniza.
    """
```


[illegible]

```

def _func_decl(self) -> FuncDecl:
    self._consume_keyword("func")
    name_tok = self._expect(TokenType.IDENTIFIER,
                             "expected function name after 'func'")
    self._expect_delim("(", "expected '(' after function name")

    params: list[str] = []
    if not self._check_delim(")"):
        params.append(self._expect(
            TokenType.IDENTIFIER, "expected parameter name").value)
        while self._match_delim(","):
            params.append(self._expect(
                TokenType.IDENTIFIER,
                "expected parameter name after ','").value)
    self._expect_delim(")", "expected ')' after parameter list")

    body = self._block()
    return FuncDecl(name=name_tok.value, params=params, body=body)

def _if_stmt(self) -> IfStmt:
    self._consume_keyword("if")
    condition = self._expression()
    then_block = self._block()
    else_block = None
    if self._check_keyword("else"):
        self._advance()
        else_block = self._block()
    return IfStmt(condition=condition,
                  then_block=then_block,
                  else_block=else_block)

def _while_stmt(self) -> WhileStmt:
    self._consume_keyword("while")
    condition = self._expression()
    body = self._block()
    return WhileStmt(condition=condition, body=body)

def _return_stmt(self) -> ReturnStmt:
    self._consume_keyword("return")
    value = None
    if not self._check_delim(";"):
        value = self._expression()
    self._expect_delim(";", "expected ';' after return statement")
    return ReturnStmt(value=value)

def _print_stmt(self) -> PrintStmt:
    self._consume_keyword("print")
    self._expect_delim("(", "expected '(' after 'print'")
    value = self._expression()
    self._expect_delim(")", "expected ')' after print argument")
    self._expect_delim(";", "expected ';' after print statement")
    return PrintStmt(value=value)

def _assignment(self) -> Assignment:
    name_tok = self._expect(TokenType.IDENTIFIER,
                             "expected identifier on left of '='")
    self._expect_operator("=", "expected '=' in assignment")
    value = self._expression()
    self._expect_delim(";", "expected ';' after assignment")
    return Assignment(name=name_tok.value, value=value)

def _expr_stmt(self) -> ExprStmt:

```

```

expr = self._expression()
self._expect_delim(";", "expected ';' after expression")
return ExprStmt(expr=expr)

def _block(self) -> Block:
    self._expect_delim("{", "expected '{' at start of block")
    stmts: list = []
    while not self._check_delim("}") and not self._is_at_end():
        try:
            stmts.append(self._statement())
        except _ParseError:
            self._synchronize()
    self._expect_delim("}", "expected '}' at end of block")
    return Block(statements=stmts)

# ■■■■■■■■■■■■■■■■■■■■■■ Expresiones (con precedencia) ■■■■■■■■■■■■■■■■■■■■■■

def _expression(self):
    return self._equality()

def _equality(self):
    left = self._comparison()
    while self._check_operator("==") or self._check_operator("!="):
        op = self._advance().value
        right = self._comparison()
        left = BinaryOp(op=op, left=left, right=right)
    return left

def _comparison(self):
    left = self._term()
    while (self._check_operator(">") or self._check_operator("<")
           or self._check_operator(">=") or self._check_operator("<=")):
        op = self._advance().value
        right = self._term()
        left = BinaryOp(op=op, left=left, right=right)
    return left

def _term(self):
    left = self._factor()
    while self._check_operator("+") or self._check_operator("-"):
        op = self._advance().value
        right = self._factor()
        left = BinaryOp(op=op, left=left, right=right)
    return left

def _factor(self):
    left = self._unary()
    while self._check_operator("**") or self._check_operator("/"):
        op = self._advance().value
        right = self._unary()
        left = BinaryOp(op=op, left=left, right=right)
    return left

def _unary(self):
    if self._check_operator("-"):
        op = self._advance().value
        operand = self._unary()
        return UnaryOp(op=op, operand=operand)
    return self._call()

def _call(self):
    expr = self._primary()
    # Sólo se permiten llamadas sobre identificadores en CalcScript

```

```

if isinstance(expr, Identifier) and self._check_delim("("):
    self._advance() # consumir '('
    args: list = []
    if not self._check_delim(")"):
        args.append(self._expression())
        while self._match_delim(","):
            args.append(self._expression())
        self._expect_delim(")", "expected ')' after call arguments")
    return Call(callee=expr.name, args=args)
return expr

def _primary(self):
    tok = self._peek()
    if tok.type == TokenType.NUMBER_INT:
        self._advance()
        return IntLiteral(value=int(tok.value))
    if tok.type == TokenType.NUMBER_FLOAT:
        self._advance()
        return FloatLiteral(value=float(tok.value))
    if tok.type == TokenType.STRING:
        self._advance()
        # quitar comillas externas
        return StringLiteral(value=tok.value[1:-1])
    if tok.type == TokenType.IDENTIFIER:
        self._advance()
        return Identifier(name=tok.value)
    if tok.type == TokenType.DELIMITER and tok.value == "(":
        self._advance()
        inner = self._expression()
        self._expect_delim(")", "expected ')' after expression")
        return inner
    raise self._error(tok, f"expected expression, got '{tok.value}'")

# ■■■■■■ Utilidades / consumo de tokens ■■■■■■

def _peek(self) -> Token:
    if self.pos < len(self.tokens):
        return self.tokens[self.pos]
    # Token sentinela EOF para evitar IndexError
    last_line = self.tokens[-1].line if self.tokens else 1
    return Token(index=-1, type=TokenType.EOF, value="<eof>",
                line=last_line, column=0)

def _peek_at(self, idx: int) -> Token | None:
    if 0 <= idx < len(self.tokens):
        return self.tokens[idx]
    return None

def _is_at_end(self) -> bool:
    return self.pos >= len(self.tokens)

def _advance(self) -> Token:
    tok = self._peek()
    if not self._is_at_end():
        self.pos += 1
    return tok

# ■■ Verificadores (no consumen) ■■
def _check_delim(self, value: str) -> bool:
    t = self._peek()
    return t.type == TokenType.DELIMITER and t.value == value

def _check_operator(self, value: str) -> bool:
```

```

        t = self._peek()
        return t.type == TokenType.OPERATOR and t.value == value

def _check_keyword(self, value: str) -> bool:
    t = self._peek()
    return t.type == TokenType.KEYWORD and t.value == value

# ■■ Matchers (consumen si hay match) ■■
def _match_delim(self, value: str) -> bool:
    if self._check_delim(value):
        self._advance()
        return True
    return False

# ■■ Consumers (consumen o reportan error) ■■
def _expect(self, tok_type: TokenType, message: str) -> Token:
    t = self._peek()
    if t.type == tok_type:
        return self._advance()
    raise self._error(t, message)

def _expect_delim(self, value: str, message: str) -> Token:
    if self._check_delim(value):
        return self._advance()
    raise self._error(self._peek(), message)

def _expect_operator(self, value: str, message: str) -> Token:
    if self._check_operator(value):
        return self._advance()
    raise self._error(self._peek(), message)

def _consume_keyword(self, value: str) -> Token:
    if self._check_keyword(value):
        return self._advance()
    raise self._error(self._peek(), f"expected keyword '{value}'")

# ■■ Reporte de error + recuperación ■■
def _error(self, tok: Token, message: str) -> _ParseError:
    self.errors.append(SyntaxError_(
        line=tok.line, column=tok.column, message=message,
    ))
    return _ParseError()

def _synchronize(self) -> None:
    """Avanza hasta el siguiente punto seguro para retomar el parseo.

    Salta tokens hasta encontrar un ';' (fin de sentencia) o una palabra
    clave que marca inicio de sentencia (func, if, while, return, print).
    Esto evita que un solo error en cascada genere docenas de errores
    falsos.
    """
    while not self._is_at_end():
        t = self._peek()
        if t.type == TokenType.DELIMITER and t.value == ";":
            self._advance()
            return
        if t.type == TokenType.KEYWORD and t.value in _STMT_KEYWORDS:
            return
        self._advance()

```

Pipeline: lectura $\rightarrow$ lexer $\rightarrow$ parser $\rightarrow$ salida.

```
# ■■■ main.py
# Punto de entrada de CalcScript.
# Uso: python main.py <archivo.csc>
#
# Pipeline de procesamiento:
# 1. Leer el archivo fuente .csc
# 2. Análisis LÉXICO → produce lista de tokens + errores léxicos
# 3. Análisis SINTÁCTICO → produce AST + errores sintácticos
# 4. Imprimir en orden: tokens, AST, errores léxicos, errores sintácticos

import sys

from lexer import Lexer
from parser import Parser
from ast_nodes import ast_to_string

def main() -> None:
    if len(sys.argv) != 2:
        print("Uso: python main.py <archivo.csc>")
        sys.exit(1)

    source_path = sys.argv[1]

    try:
        with open(source_path, encoding="utf-8") as fh:
            source = fh.read()
    except FileNotFoundError:
        print(f"Error: no se encontró el archivo '{source_path}'")
        sys.exit(1)
    except OSError as exc:
        print(f"Error al leer '{source_path}': {exc}")
        sys.exit(1)

    # ■■ Fase 1: análisis léxico ■■
    lexer = Lexer(source)
    tokens, lex_errors = lexer.tokenize()

    # ■■ Salida: lista de tokens ■■
    print()
    print("Token List:")
    for tok in tokens:
        print(f" {tok}")

    # ■■ Fase 2: análisis sintáctico ■■
    parser = Parser(tokens)
    ast, syntax_errors = parser.parse()

    # ■■ Salida: AST (representación intermedia) ■■
    print()
    print("AST (Abstract Syntax Tree):")
    print(ast_to_string(ast, indent=1))

    # ■■ Salida: errores léxicos ■■
    print()
    if lex_errors:
        print("Lexical Error List:")
        for err in lex_errors:
```

```
        print(f" {err}")
    else:
        print("No lexical errors found.")

    # ■■ Salida: errores sintácticos ■■
    print()
    if syntax_errors:
        print("Syntax Error List:")
        for err in syntax_errors:
            print(f" {err}")
    else:
        print("No syntax errors found.")

if __name__ == "__main__":
    main()
```

5. Diagrama del Autómata Léxico

Autómata finito determinista (DFA) del lexer. Nodos verdes = estados de aceptación. Nodo rojo = error. Flechas punteadas = retorno al estado inicial.

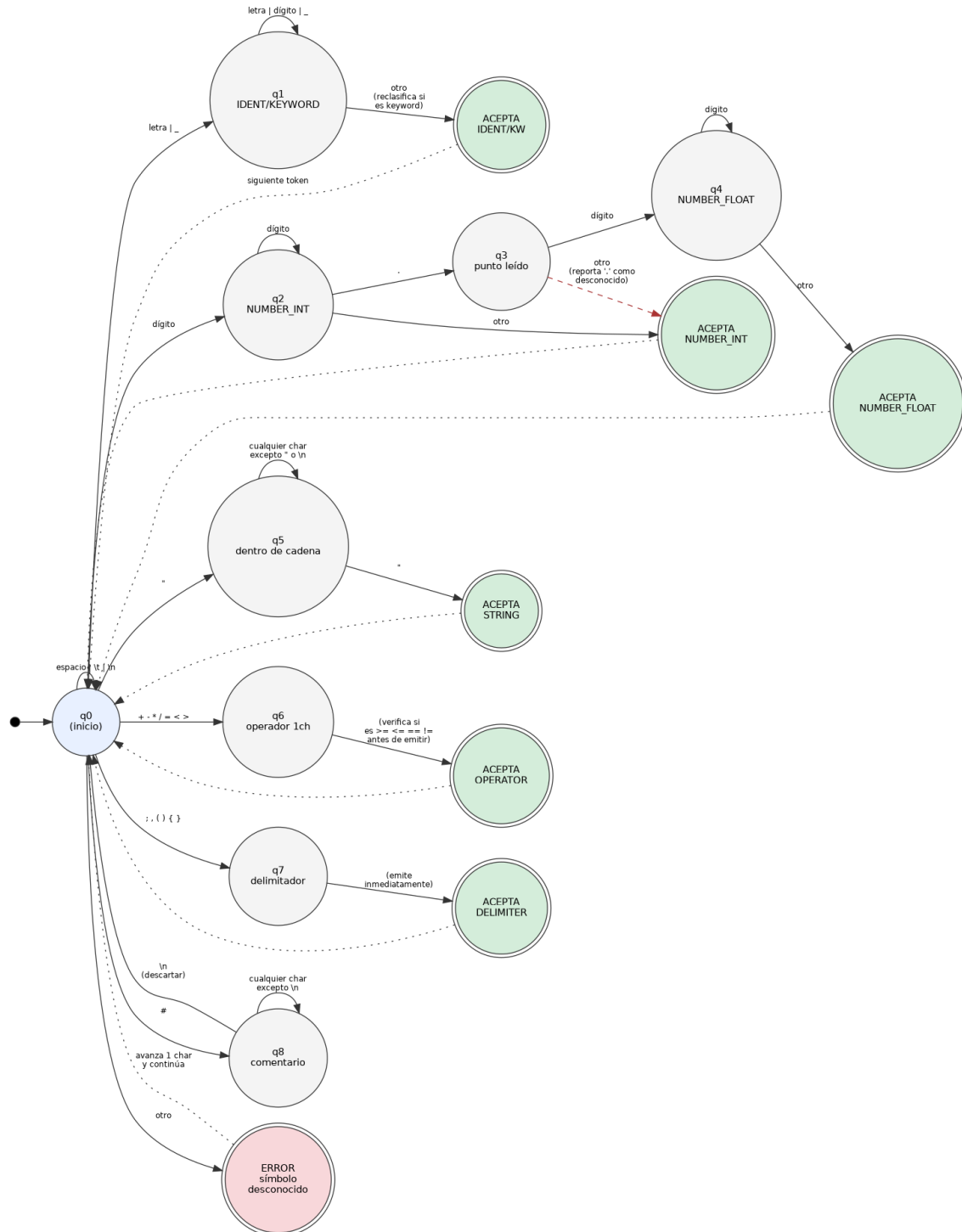


Figura 5.1 — DFA del analizador léxico de CalcScript.

6. Demostración Funcional

A continuación se presentan capturas de pantalla reales de la ejecución del programa en Windows 11, VS Code con Python 3.12.10. Cada captura muestra la salida completa: lista de tokens, AST, errores léxicos y errores sintácticos.

6.1 Programa válido: factorial.csc

Programa sin errores. El lexer produce 44 tokens y el parser construye el AST completo con FuncDecl, WhileStmt, Assignment, Call y PrintStmt.

Captura — Lista de tokens:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\USER\Desktop\calcscrip-lexer> python src/main.py tests/valid/factorial.csc

Token List:
1: (KEYWORD, "func")
2: (IDENTIFIER, "factorial")
3: (DELIMITER, "(")
4: (IDENTIFIER, "n")
5: (DELIMITER, ")")
6: (DELIMITER, "{")
7: (IDENTIFIER, "result")
8: (OPERATOR, "=")
9: (NUMBER_INT, "1")
10: (DELIMITER, ";")
11: (KEYWORD, "while")
12: (IDENTIFIER, "n")
13: (OPERATOR, ">")
14: (NUMBER_INT, "1")
15: (DELIMITER, "{")
16: (IDENTIFIER, "result")
17: (OPERATOR, "=")
18: (IDENTIFIER, "result")
19: (OPERATOR, "*")
20: (IDENTIFIER, "n")
21: (DELIMITER, ";")
22: (IDENTIFIER, "n")
23: (OPERATOR, "=")
24: (IDENTIFIER, "n")
25: (OPERATOR, "-")
26: (NUMBER_INT, "1")
27: (DELIMITER, ";")
28: (DELIMITER, "}")
29: (KEYWORD, "return")
30: (IDENTIFIER, "result")
31: (DELIMITER, ";")
32: (DELIMITER, "}")
33: (IDENTIFIER, "x")
34: (OPERATOR, "=")
35: (IDENTIFIER, "factorial")
36: (DELIMITER, "(")
37: (NUMBER_INT, "5")
38: (DELIMITER, ")")
39: (DELIMITER, ";")
40: (KEYWORD, "print")
41: (DELIMITER, "(")
42: (IDENTIFIER, "x")
43: (DELIMITER, ")")
44: (DELIMITER, ";")

AST (Abstract Syntax Tree):
0 0 0
```

Captura — AST generado:

```
AST (Abstract Syntax Tree):
  Program
    FuncDecl(name='factorial', params=[n])
      Block
        Assignment(name='result')
          IntLiteral(1)
        WhileStmt
          condition:
            BinaryOp(op='>')
              Identifier('n')
              IntLiteral(1)
          body:
            Block
              Assignment(name='result')
                BinaryOp(op='*')
                  Identifier('result')
                  Identifier('n')
              Assignment(name='n')
                BinaryOp(op='-')
                  Identifier('n')
                  IntLiteral(1)
            ReturnStmt
              Identifier('result')
        Assignment(name='x')
          Call(callee='factorial')
            IntLiteral(5)
        PrintStmt
          Identifier('x')

No lexical errors found.

No syntax errors found.
PS C:\Users\USER\Desktop\calcscrip-lexer> 
```

Captura 6.1 — 44 tokens + AST completo. Sin errores léxicos ni sintácticos.

6.2 Errores léxicos: errores_simbolos.csc

Contiene \$ y @. El lexer los reporta, el parser genera AST parcial.

```
PS C:\Users\USER\Desktop\calcscrip-lexer> python src/main.py tests/invalid/errores_simbolos.csc
```

Token List:

```
1: (IDENTIFIER, "x")
2: (OPERATOR, "=")
3: (NUMBER_INT, "10")
4: (NUMBER_INT, "2")
5: (DELIMITER, ";")
6: (IDENTIFIER, "y")
7: (OPERATOR, "=")
8: (IDENTIFIER, "total")
9: (OPERATOR, "+")
10: (NUMBER_INT, "1")
11: (DELIMITER, ";")
12: (IDENTIFIER, "z")
13: (OPERATOR, "=")
14: (IDENTIFIER, "x")
15: (OPERATOR, "!=")
16: (IDENTIFIER, "y")
17: (DELIMITER, ";")
18: (KEYWORD, "print")
19: (DELIMITER, "(")
20: (IDENTIFIER, "z")
21: (DELIMITER, ")")
22: (DELIMITER, ";")
```

AST (Abstract Syntax Tree):

```
Program
  Assignment(name='y')
    BinaryOp(op='+')
      Identifier('total')
      IntLiteral(1)
  Assignment(name='z')
    BinaryOp(op='!=')
      Identifier('x')
      Identifier('y')
  PrintStmt
    Identifier('z')
```

Lexical Error List:

```
Line 1, Column 8: Unknown symbol '$' detected
Line 2, Column 5: Unknown symbol '@' detected
```

Syntax Error List:

```
Line 1, Column 10: expected ';' after assignment
```

Captura 6.2 — Errores léxicos detectados con línea/columna. AST parcial construido.

6.3 Errores sintácticos: errores_sintaxis.csc

Programa léxicamente válido que viola la gramática: falta ;, paréntesis sin cerrar, bloque abierto.

```

PS C:\Users\USER\Desktop\calcscrip-lexer> python src/main.py tests/invalid/errores_sintaxis.csc

Token List:
 1: (IDENTIFIER, "x")
 2: (OPERATOR, "=")
 3: (NUMBER_INT, "5")
 4: (IDENTIFIER, "y")
 5: (OPERATOR, "=")
 6: (DELIMITER, "(")
 7: (NUMBER_INT, "10")
 8: (OPERATOR, "+")
 9: (NUMBER_INT, "2")
10: (DELIMITER, ";")
11: (KEYWORD, "if")
12: (IDENTIFIER, "x")
13: (OPERATOR, ">")
14: (IDENTIFIER, "y")
15: (DELIMITER, "{")
16: (KEYWORD, "print")
17: (DELIMITER, "(")
18: (IDENTIFIER, "x")
19: (DELIMITER, ")")
20: (DELIMITER, ";")

AST (Abstract Syntax Tree):
Program

No lexical errors found.

Syntax Error List:
Line 2, Column 1: expected ';' after assignment
Line 4, Column 0: expected '}'

```

Captura 6.3 — Sin errores léxicos. Errores sintácticos reportados por el parser.

6.4 Errores mixtos: errores_mixtos.csc

Combina identificador con dígito (123abc), flotante malformado (3.) y comentario. Errores léxicos y sintácticos reportados por separado.

```

PS C:\Users\USER\Desktop\calcscrip-lexer> python src/main.py tests/invalid/errores_mixtos.csc

Token List:
1: (NUMBER_INT, "123")
2: (IDENTIFIER, "abc")
3: (OPERATOR, "=")
4: (NUMBER_INT, "5")
5: (DELIMITER, ";")
6: (IDENTIFIER, "result")
7: (OPERATOR, "=")
8: (NUMBER_INT, "3")
9: (OPERATOR, "+")
10: (NUMBER_INT, "2")
11: (DELIMITER, ";")
12: (IDENTIFIER, "x")
13: (OPERATOR, "=")
14: (IDENTIFIER, "total")
15: (KEYWORD, "print")
16: (DELIMITER, "(")
17: (IDENTIFIER, "x")
18: (DELIMITER, ")")
19: (DELIMITER, ";")

AST (Abstract Syntax Tree):
Program
  Assignment(name='result')
    BinaryOp(op='+')
      IntLiteral(3)
      IntLiteral(2)
  PrintStmt
    Identifier('x')

Lexical Error List:
Line 1, Column 1: Invalid identifier: identifiers cannot start with a digit near '1'
Line 2, Column 11: Unknown symbol '.' detected

Syntax Error List:
Line 1, Column 4: expected ';' after expression
Line 4, Column 1: expected ';' after assignment
PS C:\Users\USER\Desktop\calcscrip-lexer>

```

Captura 6.4 — 2 errores léxicos + 2 errores sintácticos reportados por separado.

7. Conclusiones

Aprendizajes individuales

Romero Guzmán Martín Gastón. Diseñar la tabla de tokens y las expresiones regulares me enseñó por qué el orden de evaluación es crítico: mover `NUMBER_INT` antes que `NUMBER_FLOAT` causaba que `3.14` se tokenizara como tres piezas. Implementar el parser por descenso recursivo me mostró que la precedencia de operadores se codifica en la jerarquía de funciones (`equality` → `comparison` → `term` → `factor`), sin tablas externas. La recuperación de errores mediante sincronización (*panic mode*) fue la parte más retadora pero también la más satisfactoria.

Mar Saucedo Duilio Alessandro. Mi contribución principal fue la definición de los nodos del AST y el pretty-printer. Separar cada tipo de nodo en su propia dataclass hizo que el parser fuera fácil de probar. También trabajé en los archivos de prueba con errores, donde aprendí que un buen lexer no se cae al encontrar un símbolo desconocido: lo reporta y continúa. La separación modular en seis archivos facilitó el trabajo en paralelo. Construir el diagrama del autómata me ayudó a visualizar formalmente las transiciones del lexer.

Dificultades encontradas

El problema más sutil fue el flotante malformado `3.`: el lexer matchea `3` como `NUMBER_INT` y reporta `.` como símbolo desconocido. Los errores léxicos propagan incoherencia al parser, por lo que se reportan por separado.

Trabajo futuro

- Análisis semántico (tipos, alcance de variables).
- Intérprete que evalúe el AST.
- Cadenas multilínea y notación científica.
- Código intermedio de tres direcciones (TAC).
- Exportar AST como JSON o gráfico Graphviz.

8. Referencias

- [1] Aho, A. V. et al. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson.
- [2] Cooper, K. D. y Torczon, L. (2011). *Engineering a Compiler* (2.^a ed.). Morgan Kaufmann.
- [3] Nystrom, R. (2021). *Crafting Interpreters*. <https://craftinginterpreters.com>
- [4] Python Software Foundation. (2024). *re — Regular expression operations*. <https://docs.python.org/3/library/re.html>
- [5] Python Software Foundation. (2024). *dataclasses*. <https://docs.python.org/3/library/dataclasses.html>
- [6] Levine, J. (2009). *Flex & Bison*. O'Reilly Media.
- [7] Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.